

Spam Email Classification Using Decision Trees and XGBoost with Cross-Validation

Machine Learning Project — concise classification pipeline for email.

Team:

N. Yishnaani — 2520030414

K. Pragnya — 2520030430

Guide: Shaik Asif, Assistant Professor, CSE Dept. — Academic Year: 2026–27

Visual: Email → Machine Learning → SPAM / LEGITIMATE

Why Do We Need Spam Detection?

- **Email is ubiquitous.** Volume makes manual filtering impractical.
- **Spam harms users.** Phishing, scams, and unwanted content.
- **ML learns patterns** from labeled emails to classify new messages.
- **Automation saves time** and reduces user exposure to threats.

Problem Statement

Develop a simple ML system to classify emails as **Spam** or **Legitimate**.

Objectives & Basic Approach

- **Prepare** and clean the email dataset.
- **Convert** text into numerical features (TF-IDF).
- **Train** multiple ML models and compare.
- **Evaluate** with cross-validation and metrics.
- **Identify** effectiveness of Decision Tree and XGBoost.

Visual summary: Dataset → Preprocessing → Feature Extraction → Model Training → Cross-Validation → Evaluation → Prediction

Data Preprocessing & TF-IDF

Step 1 — Preprocessing

- **Clean text:** remove HTML, headers, punctuation, and non-text noise.
- **Normalize:** lowercasing, tokenization, optional stopword removal.
- **Result:** processed plain-text ready for feature extraction.

Step 2 — TF-IDF

TF-IDF converts processed text to numerical features by weighting terms by importance across the corpus.



Logistic Regression

Baseline model.

Linear separation of classes. Fast and interpretable.



Decision Tree

Tree-based. Rules split features to decide Spam vs Legitimate. Easy to visualize.



Random Forest

Ensemble of trees. Reduces variance by averaging many trees.

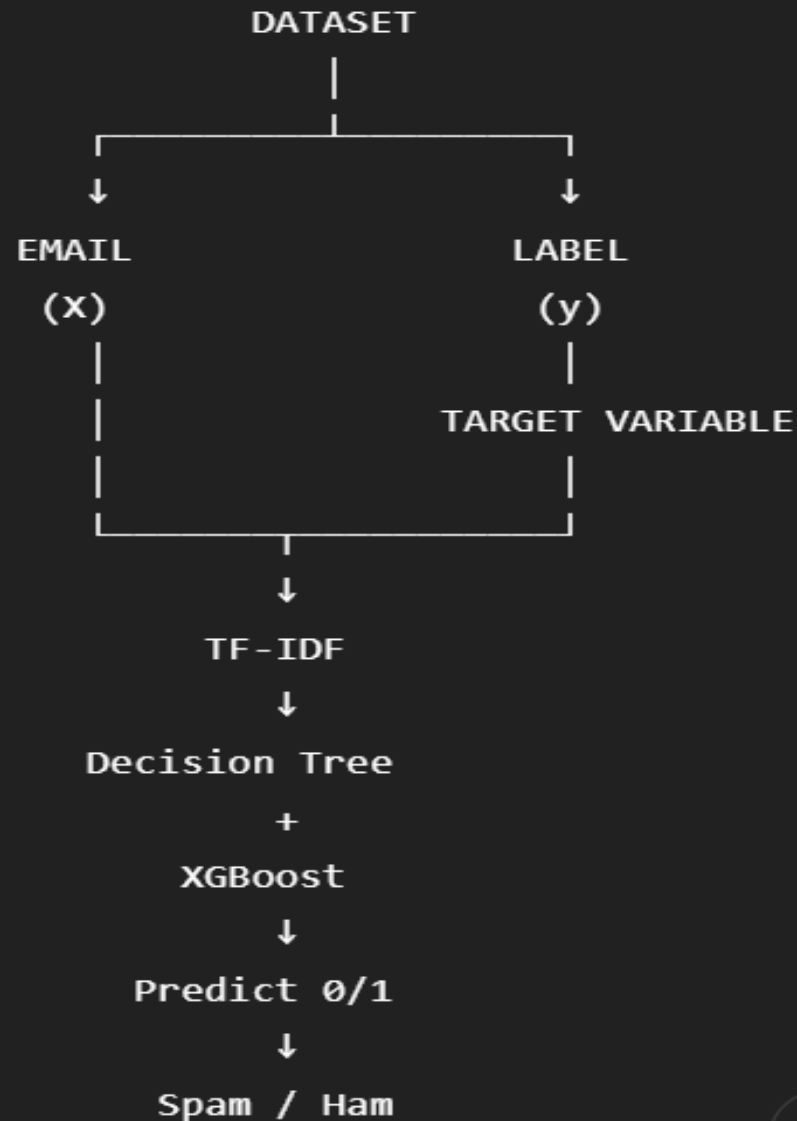


XGBoost

Boosting model. Sequential trees improve errors; often strong performance.

Goal: Compare models using the same data and validation process. No claims about which is best without results.

FLOW OF THE SPAM EMAIL DETECTION



Existing problem

↓
Spam emails are constantly changing

↓
Build ML-based classifier

↓
Convert email text → TF-IDF numerical features

↓
Train two different models

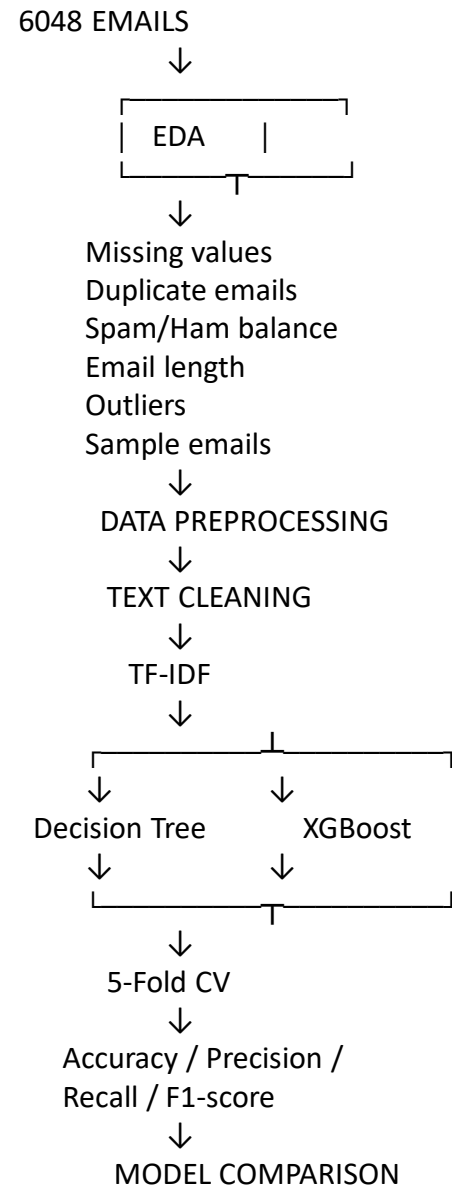
↓
Decision Tree

XGBoost

↓
Compare performance

↓
Cross-validation

↓
Accuracy / Precision /
Recall / F1-score



Decision Tree vs XGBoost — Key Differences

Decision Tree

Single model with branch decisions. Simple splits and easy to explain.

XGBoost

Sequential ensemble of trees using gradient boosting. Focuses on correcting previous errors.

Interpretability

Decision Tree: **high**. XGBoost: **lower** but can be explained with SHAP (optional).

Cross-Validation & Hyperparameter Tuning

- **Avoid memorization.** Do not let models simply memorize training examples.
- **Cross-validation** tests models on multiple data portions for robust estimates.
- **Hyperparameter tuning** searches for settings that improve generalization.
- **Final evaluation** uses held-out data after tuning for an unbiased check.

Evaluation Metrics — What to Measure



Accuracy

Overall fraction of correct predictions.



Precision

How many predicted spam are actually spam.



Recall

How much actual spam the model detects.



F1-Score

Harmonic mean of precision and recall.



ROC-AUC & PR-AUC

Separation ability and precision-recall tradeoffs.



Calibration

Whether predicted probabilities reflect real confidence.

Unsupervised Learning & Data Exploration

- **K-Means:** find similar groups of emails.
- **DBSCAN:** detect dense clusters and anomalies.
- **PCA:** reduce dimensions for simpler views.
- **t-SNE / UMAP:** visualize high-dimensional patterns.

Purpose: Discover patterns and anomalies separate from the supervised task.

Conclusion

- Built a simple ML pipeline for **Spam vs Legitimate** classification.
- Used **preprocessing** and **TF-IDF** to create features from email text.
- Compared models including **Decision Tree** and **XGBoost**.
- Applied **cross-validation** and multiple metrics for reliable assessment.
- Used unsupervised methods to **explore** patterns and anomalies.

Thank You

Questions?